

Karam mahameed – 213029143

Moamen haj yosef - 212489942

מעבדה 2 חלק ב

1. תיאור הבעיה:

בפרק זה מימשנו פתרון לבעיית $minmax TSP$ הבעיה מבוססת על בעיית הסוכן הנוסע הרגילה אבל במקום למצוא מסלול אחד קצר צריך למצוא שני מסלולים ***Hamiltonian*** שונים על אותו סט ערים. כל מסלול חייב לעבור בכל הערים בדיוק פעם אחת ולחזור לעיר ההתחלה. בנוסף קיימת מגבלה שאסור שתהיה חפיפה בין המסלולים ברמת הקשתות.

מטרת הבעיה היא למזער את אורך המסלול הארוך מבין שני המסלולים:

$$score = \max(c(T1), c(T2))$$

כלומר לא מספיק שהסכום הכולל של שני המסלולים יהיה קטן. חשוב במיוחד שהמסלול הארוך יותר יהיה כמה שיותר קצר, כי הוא זה שקובע את ערך הפתרון.

• ייצוג הפתרון:

בחרנו לייצג כל פתרון כזוג של תמורות:

$$route1 = \text{permutation of cities}$$

$$route2 = \text{permutation of cities}$$

כל תמורה מייצגת מסלול *Hamiltonian* אחד. אם יש n ערים כל מסלול מכיל את כל הערים מ 0 עד $n - 1$ בדיוק פעם אחת.

הפתרון נחשב חוקי רק אם $overlaps = 0$

אם יש קשתות חופפות הפתרון מקבל ענישה גדולה מאוד ב $fitness$ כך האלגוריתם מעדיף פתרונות חוקיים.

• במימוש השתמשנו בכמה קונפיגורציות שונות:

- *regular_ga* : בגרסה זו האלגוריתם משתמש רק במנוע הגנטי הרגיל:
initial population, selection, crossover, mutation, repair

repair, elitsim, fitness evaluation

לא מופעל חיפוש מקומי בו. מטרת גרסה זו היא לשמש *baseline* ולבדוק מה האלגוריתם הגנטי מצליח להשיג לבד.

- *partial_memtic* : בגרסה זו האלגוריתם זהה ל *regular_ga* אבל נוסף חיפוש מקומי חלקי. החיפוש המקומי לא מופעל על כל פרט ולא בכל דור במימוש שלנו, החיפוש המקומי מופעל בצורה חלקית לפי ההגדרות:

local_search_mode = partial

local_search_every = 5

local_search_rate = 0.25

כלומר כל כמה דורות חלק מהילדים יכולים לעבור שיפור מקומי. מטרת גרסה זו היא לבדוק האם אפשר לקבל שיפור באיכות הפתרון בלי לשלם זמן ריצה גבוה מדי.

- *full_memtic* : בגרסה זו מופעל חיפוש מקומי על כל ילד חדש שנוצר כלומר לאחר *crossover, mutation, repair* הילד עובר *local search* לפני שהוא נכנס לאוכלוסייה.

במימוש שלנו החיפוש המקומי מבוסס על $opt - 2$ ומתחיל קודם מהמסלול הארוך יותר כי הוא זה שקובע את ערך ה *minimax*. גרסה זו צפויה לתת פתרונות טובים יותר אך זמן הריצה שלה גבוה יותר.

Local search : בגרסאות הממטיות הוספנו חיפוש מקומי מסוג $opt - 2$ הפעולה של $opt - 2$ בוחרת מקטע בתוך המסלול והופכת אותו. החיפוש המקומי מופעל קודם על המסלול הארוך יותר. הסיבה היא שהמטרה היא למזער $score = \max(c(T1), c(T2))$ ולכן המסלול הארוך יותר הוא צוואר הבקבוק של הפתרון. שינוי מתקבל רק אם הפתרון נשאר חוקי, אין קשתות חופפות. ערך ה *score* משתפר.

2. דוגמת הרצה:

ניתן להריץ את התוכנית עם seeds מסוים על כל התיקייה נקראת tsps שבתוכה כל קבצי קלט tsp.

```
C:\Users\karam\Desktop\Workshop in AI\Minmax_TSP>py main.py --folder "tsps" --method all --seeds 8
```

אפשר גם להריץ התוכנית על קלט עבור רק קונפגרציה מסוימת:

```
:\Users\karam\Desktop\Workshop in AI\Minmax_TSP>py main.py --folder "tsps" --method regular_ga --seeds 7
```

```
C:\Users\karam\Desktop\Workshop in AI\Minmax_TSP>py main.py --folder "tsps" --method partial_memetic --seeds 7
```

```
C:\Users\karam\Desktop\Workshop in AI\Minmax_TSP>py main.py --folder "tsps" --method full_memetic --seeds 7
```

או על קובץ קלט יחיד לדוגמה, עבור *burma14.tsp*

```
C:\Users\karam\Desktop\Workshop in AI\Minmax_TSP>py main.py --tsp "tsps/burma14.tsp" --method all --seeds 7
```

התוצאות שנראות למטה גם נשמרות בתיקייה בשם results עבור כל קונפגרציה נשמר קובץ בשם הקונפגרציה ושם הקלט:

```
Instance: burma14
Method: regular_ga

Route 1:
13 3 4 5 12 6 7 14 2 1 8 11 10 9

Route 2:
8 13 7 5 6 4 12 14 3 2 10 1 11 9

Route 1 cost: 3756
Route 2 cost: 3894
Best score max(c(T1), c(T2)): 3894
Total cost c(T1)+c(T2): 7650
Balance |c(T1)-c(T2)|: 138
Legal solution: True
Overlapping edges: 0

Runtime: 1.1445
Generations to convergence: 130
Legal solution rate: 1.0000
Diversity: 0.2235
Seed: 7

burma14      | regular_ga      | seed= 7 | score= 3894.00 | c1= 3756.00 | c2= 3894.00 | legal=True | overlaps=0 | time=1.14s
```

```

Instance: burma14
Method: partial_memetic

Route 1:
5 6 12 4 3 8 2 1 10 9 11 13 14 7

Route 2:
1 9 13 7 6 4 5 12 14 3 2 10 11 8

Route 1 cost: 3807
Route 2 cost: 3838
Best score max(c(T1), c(T2)): 3838
Total cost c(T1)+c(T2): 7645
Balance |c(T1)-c(T2)|: 31
Legal solution: True
Overlapping edges: 0

Runtime: 1.6090
Generations to convergence: 125
Legal solution rate: 1.0000
Diversity: 0.1915
Seed: 7

burma14      | partial_memetic | seed= 7 | score= 3838.00 | c1= 3807.00 | c2= 3838.00 | legal=True | overlaps=0 | time=1.61s

```

```

Instance: burma14
Method: full_memetic

Route 1:
7 13 8 1 9 11 10 2 14 3 12 5 4 6

Route 2:
14 13 11 8 9 10 1 2 3 4 12 6 5 7

Route 1 cost: 3770
Route 2 cost: 3785
Best score max(c(T1), c(T2)): 3785
Total cost c(T1)+c(T2): 7555
Balance |c(T1)-c(T2)|: 15
Legal solution: True
Overlapping edges: 0

Runtime: 11.5273
Generations to convergence: 65
Legal solution rate: 1.0000
Diversity: 0.2178
Seed: 7

burma14      | full_memetic    | seed= 7 | score= 3785.00 | c1= 3770.00 | c2= 3785.00 | legal=True | overlaps=0 | time=11.53s

```

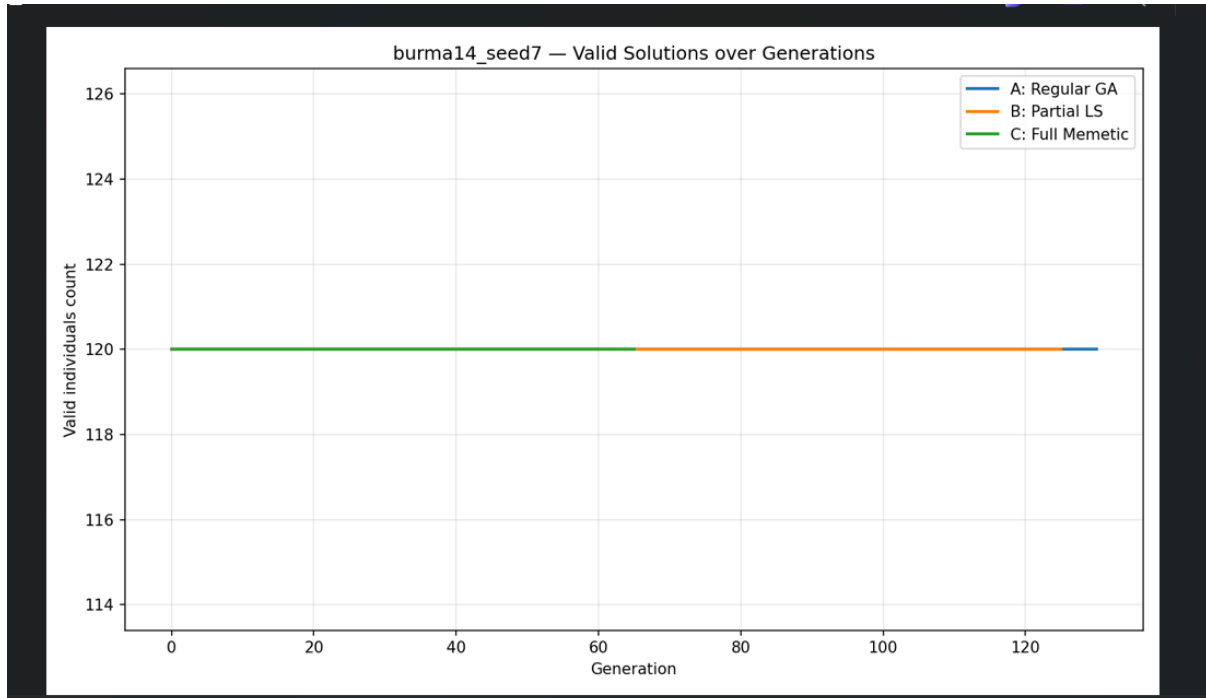
טבלת השוואות עבור ההרצה זאת:

Configuration	Route 1 cost	Route 2 cost	Minmax score	Total cost	balance
Regular_ga	3756	3894	3894	7650	138
Partial_memetic	3807	3838	3838	7645	31
Full_memetic	3770	3785	3785	7555	15

- Regular_ga : נתן פתרון חוקי ומהיר מאוד אבל נתן את Minmax score הכי גבוה מבין הקונפגרציות כלומר המסלול הארוך ביותר גם balance היה הכי חלש זה אומר ששני המסלולים לא מאוזנים כל כך מסלול אחד קצר יותר והשני גדול יותר וכך Minmax score נפגע כי מקבלים רק הארוך בסוף הוא טוב כבסיס אבל בלי local search הוא פחות מצליח לאזן בין שני המסלולים ופחות מצליח להקטין את המסלול הארוך.

- **Partial_memetic** : כאן רואים שיפור לעומת **Regular_ga** ה **Minmax** score ירד כלומר המסלול הארוך התקצר. גם **Total Cost** ירד אבל השיפור הכי ברור הוא ב **Balance** זה אומר שהחיפוש המקומי החלקי עזר לקרב את שני המסלולים אחד לשני מבחינת עלות. כלומר הפתרון הפך להיות יותר מאוזן בלי להוסיף יותר מדי זמן ריצה.
- **Full_memetic** : מקונפגרציה זו התקבלה התוצאה הכי טובה מבין שלוש הקונפיגורציות. **Minmax score** הכי נמוך וגם **Total Cost** ו- **Balance** כלומר לא רק הקטין את המסלול הארוך אלא גם הקטין את הסכום הכולל של שני המסלולים וגם יצר שני מסלולים קרובים מאוד באורך שלהם. החיפוש המקומי המלא מצליח לשפר כל ילד חדש ולכן הוא מוביל למסלולים קצרים ומאוזנים יותר החיסרון הוא זמן ריצה גבוה יותר.

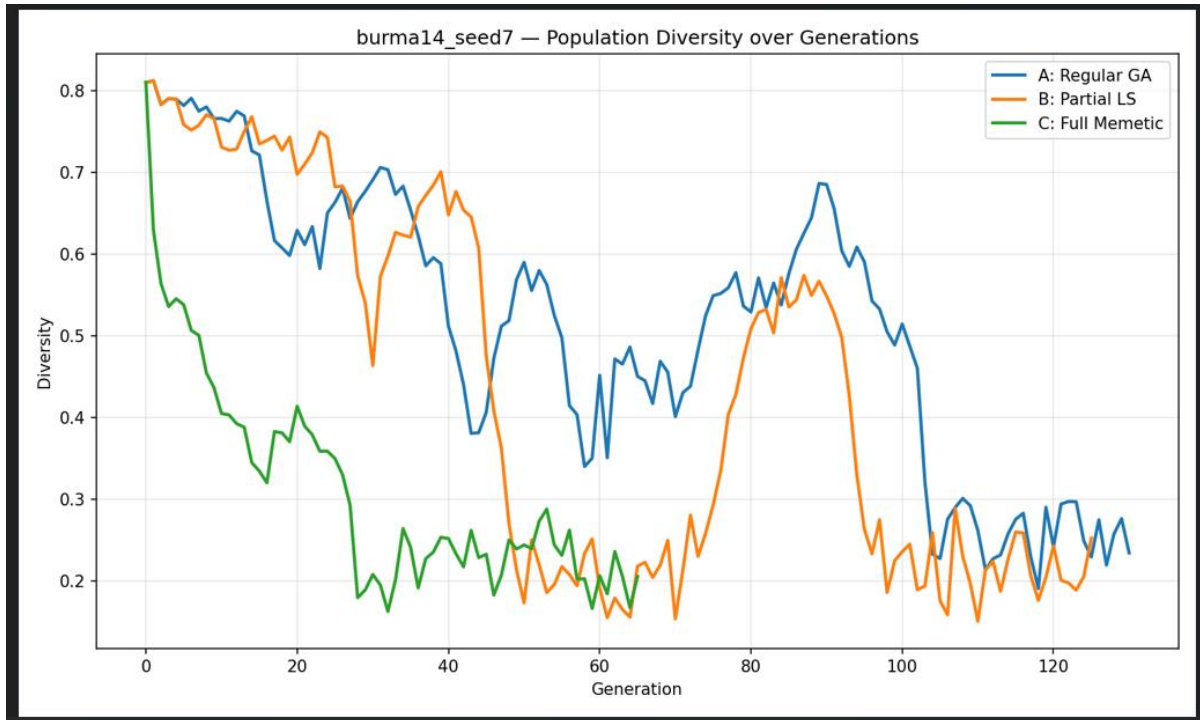
- ונוצרת תיקיית *plots* בתוך *results* שבה נשמרת הגרפים:
א. גרף *valid solutions over generations*:



הציר האופקי הוא מספר הדור והציר האנכי הוא כמה פרטים באוכלוסייה הם חוקיים. במקרה שלך גודל האוכלוסייה הוא 120 ולכן כאשר הגרף נמצא על 120 זה אומר שכל האוכלוסייה חוקית. בגרף רואים שכל שלוש הקונפיגורציות נשארות כל הזמן על 120 פתרונות חוקיים זה אומר שמנגנון ה *repair* עובד טוב מאוד כי גם אחרי *crossover & mutation* אם נוצרות קשתות חופפות בין שני המסלולים האלגוריתם מצליח לתקן את הפתרון לפני שהוא נכנס לאוכלוסייה

- ככל שיש יותר ניסיונות תיקון יש יותר סיכוי להחזיר פתרון לא חוקי להיות חוקי.
- מוטציה חזקה מדי יכולה ליצור יותר חפיפות בקשתות ולכן להוריד את מספר הפתרונות החוקיים
- *Crossover* בין שני מסלולים יכול ליצור ילדים עם קשתות משותפות ולכן גם הוא משפיע על חוקיות הפתרונות.
- *Opt-2* משנה קטעים במסלול אם לא בודקים חוקיות אחרי השינוי הוא עלול ליצור חפיפות. אצלך החיפוש המקומי מקבל שינוי רק אם נשארים עם 0 ולכן הגרף נשאר גבוה.

ב. גרף Population Diversity over Generations:

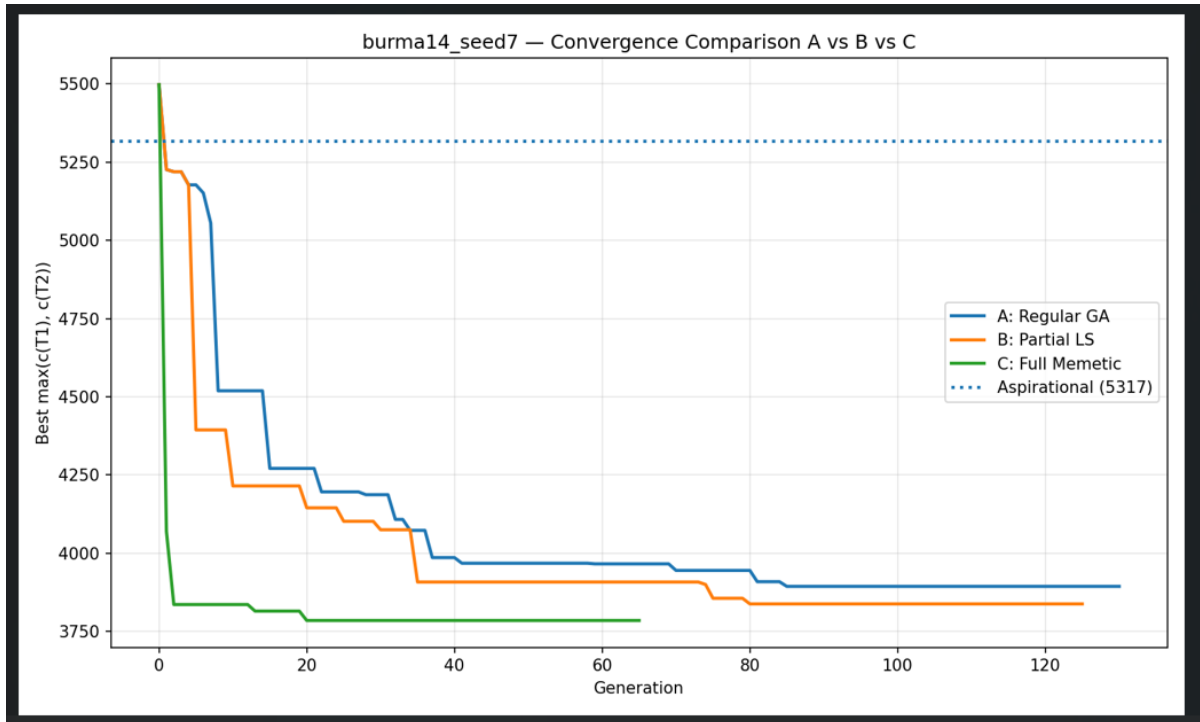


מציג כמה האוכלוסייה מגוונת לאורך הדורות. כלומר הוא בודק עד כמה הפתרונות באוכלוסייה שונים אחד מהשני לפי הקשתות שמופיעות במסלולים ערך גבוה של diversity אומר שהאוכלוסייה מכילה פתרונות שונים ומגוונים ערך נמוך אומר שהפרטים באוכלוסייה נהיו דומים אחד לשני כלומר האלגוריתם התחיל להתכנס לאותו אזור של פתרונות. בגרף רואים שבתחילת ההרצה שלוש הקונפיגורציות מתחילות עם diversity גבוה כי האוכלוסייה ההתחלתית נוצרת בצורה אקראית ולכן בהתחלה יש הרבה פתרונות שונים.

- Regular GA הקו הכחול הגיוון יורד בהדרגה אבל לא בבת אחת. זה קורה כי אין חיפוש מקומי שמושך את כל הפרטים לאותו כיוון מהר. הגיוון נשאר יחסית גבוה לאורך הרבה דורות, ויש גם עליות וירידות.

- Partial Memetic : הקו הכתום, רואים שבהתחלה הגיוון יורד אבל בצורה לא לגמרי יציבה. יש ירידות חדות ואז עליות מחדש. זה מתאים לאופי של האלגוריתם החיפוש המקומי מופעל רק בחלק מהדורות ועל חלק מהפרטים. לכן לפעמים הוא מקרב פתרונות אחד לשני ומוריד diversity.
- Full Memetic : הקו הירוק, רואים ירידה הרבה יותר מהירה בגיוון כבר בתחילת ההרצה. זה בגלל שכל ילד חדש עובר local search ולכן הרבה פתרונות שונים משתפרים לכיוונים דומים. במילים אחרות הוא עושה exploitation חזק מאוד הוא משפר פתרונות קיימים מהר אבל בגלל זה הוא גם מצמצם את הגיוון באוכלוסייה מהר יותר.

ג. גרף Convergence Comparison:



מציג איך ערך ה Minimax score משתפר לאורך הדורות בכל אחת משלוש הקונפיגורציות. הציר האופקי הוא מספר הדור והציר האנכי הוא הערך הטוב ביותר שנמצא עד אותו דור כלומר ככל שהקו נמוך יותר הפתרון טוב יותר. הקו המקווקו הוא היעד האספירטיבי כל הקונפיגורציות יורדות מתחת לקו הזה מהר מאוד ולכן כולן מצליחות לעמוד ביעד.

- Regular GA : הקו הכחול יורד בהדרגה. בהתחלה יש שיפור גדול אז השיפור נהיה איטי יותר. זה אומר ש GA מצליח למצוא פתרונות טובים בעזרת selection, crossover, mutation אבל בלי local search הוא צריך יותר דורות כדי לשפר את הפתרון. בסוף הוא נעצר בערך חוקי וטוב אבל פחות טוב מהגרסאות הממטיות.

- Partial Memetic: הקו הכתום יורד מהר יותר מ Regular GA ומגיע לערך נמוך יותר. זה קורה כי החיפוש המקומי מופעל חלקית ועוזר לשפר חלק מהפתרונות. לכן הוא מצליח להגיע ל score טוב יותר בזמן יחסית קצר. מצד שני בגלל שה local search לא מופעל על כל פרט השיפור שלו עדיין מוגבל.
- Full Memetic : הקו הירוק יורד בצורה הכי חדה כבר בתחילת הריצה. זה בגלל שכל ילד חדש עובר local search ולכן האלגוריתם משפר פתרונות בצורה אינטנסיבית מהדורות הראשונים. הוא מגיע מהר מאוד לאזור טוב ונעצר סביב שזה הפתרון הטוב ביותר מבין שלוש הקונפיגורציות החיסרון הוא שזמן הריצה שלו גבוה יותר.

3. קטעי קוד חשובות:

- ייצוג פתרון:

```
#Represents one solution candidate.
@dataclass 3 usages
class Individual:
    route1: list
    route2: list

    cost1: float = 0.0
    cost2: float = 0.0
    score: float = 0.0
    total: float = 0.0
    balance: float = 0.0
    overlaps: int = 0
    legal: bool = False
    fitness: float = 0.0
```

הקטע הזה מייצג פתרון אחד לבעיה. כל פתרון מורכב משני מסלולים 1 ו 2 כל מסלול הוא permutation של הערים כלומר כל עיר מופיעה פעם אחת בדיוק. בנוסף נשמרים ערכי המדידה של הפתרון הנדרשות במטלה. הוא חשוב כי בלי המבנה הזה אי אפשר להשוות פתרונות לשמור את הפתרון הכי טוב או להעביר פרטים בין דורות באלגוריתם הגנטי.

- קונפיגורציות האלגוריתם:

```
~ # Regular GA: no local search.
# Partial memetic: local search is applied sometimes
# Full memetic: local search is applied to every new child.
~ CONFIGS = {
    "regular_ga": GAConfig(
        name="regular_ga",
        local_search_mode="none",
        local_search_rate=0.0,
        local_search_every=999999,
        local_search_max_moves=0
    ),

    "partial_memetic": GAConfig(
        name="partial_memetic",
        local_search_mode="partial",
        local_search_rate=0.25,
        local_search_every=5,
        local_search_max_moves=40
    ),

    "full_memetic": GAConfig(
        name="full_memetic",
        local_search_mode="full",
        local_search_rate=1.0,
        local_search_every=1,
        local_search_max_moves=80
    )
}
```

הקטע הזה מגדיר את שלוש הקונפיגורציות שנדרש להשוות. הוא חשוב כי העבודה דורשת השוואה בין שלוש גרסאות כאן ההבדל המרכזי הוא מתי וכמה מפעילים local search כך אפשר להריץ את אותו מנוע GA אבל עם התנהגות שונה לכל שיטה.

- חישוב אורך מסלול:

```
def tour_length(route, dist): 4 usages
    total = 0

    for i in range(len(route)):
        a = route[i]
        b = route[(i + 1) % len(route)]
        total += dist[a][b]

    return total
```

הפונקציה מחשבת את אורך המסלול. היא עוברת על כל זוג ערים עוקבות במסלול ומוסיפה את המרחק ביניהן. גורם לכך שהעיר האחרונה תתחבר בחזרה לעיר הראשונה ולכן המסלול הוא Hamiltonian cycle. חישוב כי פונקציית המטרה תלויה באורך שני המסלולים. בלי חישוב נכון של העלות אי אפשר לדעת איזה פתרון טוב יותר.

- בדיקת קשתות חופפות:

```
def edge_set(route): 6 usages
    edges = set()

    for i in range(len(route)):
        a = route[i]
        b = route[(i + 1) % len(route)]

        if a > b:
            a, b = b, a

        edges.add((a, b))

    return edges

def count_overlaps(route1, route2): 3 usages
    return len(edge_set(route1).intersection(edge_set(route2)))
```

הפונקציה הופכת מסלול לקבוצת קשתות. כל קשת נשמרת בצורה לא מכוונת לאחר מכן `count_overlaps` בודקת כמה קשתות משותפות יש בין שני המסלולים. הוא בודק את האילוף המרכזי של הבעיה אסור שתהיה קשת משותפת בין שני המסלולים.

- פונקציית הערכה:

```
def evaluate(ind, dist, config): 4 usages
    ind.cost1 = tour_length(ind.route1, dist)
    ind.cost2 = tour_length(ind.route2, dist)

    ind.score = max(ind.cost1, ind.cost2)
    ind.total = ind.cost1 + ind.cost2
    ind.balance = abs(ind.cost1 - ind.cost2)

    ind.overlaps = count_overlaps(ind.route1, ind.route2)
    ind.legal = ind.overlaps == 0

    ind.fitness = ind.score + config.penalty_per_overlap * ind.overlaps

    return ind
```

הפונקציה מחשבת את כל הערכים החשובים של פתרון fitness כולל גם ענישה על פתרון לא חוקי זו פונקציית המטרה של האלגוריתם. היא גורמת לאלגוריתם להעדיף פתרונות עם Minimax נמוך אבל גם להתרחק מפתרונות לא חוקיים.

- השוואת פתרונות:

```
def better(a, b): 2 usages
    if a.legal != b.legal:
        return a.legal

    if a.fitness != b.fitness:
        return a.fitness < b.fitness

    if a.score != b.score:
        return a.score < b.score

    if a.total != b.total:
        return a.total < b.total

    return a.balance < b.balance
```

הפונקציה קובעת איזה פתרון עדיף האלגוריתם משתמש בפונקציה הזו כדי לבחור הורים טובים לשמור את הפתרון הכי טוב ולהחליט אם פתרון חדש טוב יותר מפתרון קודם.

- תיקון פתרונות לא חוקיים:

```
✓ def repair(ind, dist, config): 2 usages
    best_route2 = ind.route2[:]
    best_overlaps = count_overlaps(ind.route1, best_route2)
    best_cost = tour_length(best_route2, dist)
✓ if best_overlaps == 0:
    return
    current = best_route2[:]
✓ for _ in range(config.repair_attempts):
    candidate = current[:]
    ✓ if random.random() < 0.7:
        swap_mutation(candidate)
    ✓ else:
        inversion_mutation(candidate)
    candidate_overlaps = count_overlaps(ind.route1, candidate)
    candidate_cost = tour_length(candidate, dist)
✓ if candidate_overlaps < best_overlaps:
    current = candidate
    best_route2 = candidate
    best_overlaps = candidate_overlaps
    best_cost = candidate_cost
✓ elif candidate_overlaps == best_overlaps and candidate_cost < best_cost:
    current = candidate
    best_route2 = candidate
    best_cost = candidate_cost
✓ if best_overlaps == 0:
    break
ind.route2 = best_route2
```

הפונקציה מנסה לתקן פתרון לא חוקי אם שני המסלולים חולקים קשתות הפונקציה משנה את המסלול השני בעזרת mutation עד שמספר הקשתות החופפות יורד. ה חשוב מאוד כי אחרי crossover או mutation יכולים להיווצר פתרונות לא חוקיים. Repair עוזר לשמור על אוכלוסייה חוקית.

- הפעלת Local Search לפי קונפיגורציה:

```
def should_apply_local_search(config, generation): 1 usage
    if config.local_search_mode == "full":
        return True

    if config.local_search_mode == "partial":
        if generation % config.local_search_every == 0:
            return random.random() < config.local_search_rate

    return False
```

זה חשוב כי זו בדיוק ההפרדה בין שלוש הקונפיגורציות שנדרש להשוות.

● מדד גיוון:

```
def edge_similarity(ind1, ind2): 1 usage
    edges1 = edge_set(ind1.route1).union(edge_set(ind1.route2))
    edges2 = edge_set(ind2.route1).union(edge_set(ind2.route2))

    if not edges1 and not edges2:
        return 1.0

    return len(edges1.intersection(edges2)) / len(edges1.union(edges2))

def population_diversity(population, samples=80): 2 usages
    if len(population) < 2:
        return 0.0

    similarities = []

    for _ in range(samples):
        a, b = random.sample(population, k=2)
        similarities.append(edge_similarity(a, b))

    average_similarity = sum(similarities) / len(similarities)
    return 1.0 - average_similarity
```

הקוד מחשב diversity לפי דמיון בין קשתות. אם שני פתרונות משתמשים בהרבה מאותן קשתות הם דומים. אם יש מעט חפיפה בין הקשתות שלהם הם שונים. זה חשוב כי העבודה דורשת להתייחס לגיוון באוכלוסייה. הגרף של diversity מבוסס על המדד הזה.

- הלולאה המרכזית:

```
def run_ga(dist, config, seed=1): 2 usages
    """
    Run the full GA / memetic algorithm.
    """
    random.seed(seed)
    start_time = time.time()
    population = create_initial_population(dist, config)
    best = min(population, key=lambda x: x.fitness).clone()
    last_best_fitness = best.fitness
    no_improve = 0
    convergence_generation = config.generations
    # This list stores data for the graphs.
    history = []
    # Save generation 0 before evolution starts.
    history.append(history_row(generation=0, population, best))
    for generation in range(1, config.generations + 1):
        population.sort(key=lambda x: x.fitness)
        # Keep elite individuals unchanged.
```

זה תחילת האלגוריתם. כאן קובעים seed כדי שהריצה תהיה ניתנת לשחזור יוצרים אוכלוסייה התחלתית ושומרים את הפתרון הכי טוב. חשוב כי משפיע על כל ההחלטות האקראיות ולכן מאפשר להשוות תוצאות בצורה מסודרת.

- עצירה והתכנסות:

```
# Track improvement for convergence.
if best.fitness < last_best_fitness:
    last_best_fitness = best.fitness
    no_improve = 0
else:
    no_improve += 1
# Early stop if there is no improvement for many
if no_improve >= config.convergence_patience:
    convergence_generation = generation
    break
```

הקטע הזה בודק אם יש שיפור. אם אין שיפור במשך מספר דורות רצופים האלגוריתם עוצר מוקדם. אין טעם להמשיך להריץ אם האלגוריתם כבר התכנס ולא משתפר. זה מקצר זמן ריצה

4. תוצאות עבור קבצי tsp עם 7 seed:

• Eli76.tsp:

Balance	Total cost	Minimax score	Route 2 cost	Route 1 cost	Configuration
17	2073	1045	1028	1045	regular_ga
5	1649	827	822	827	partial_memetic
6	1378	692	686	692	full_memetic

• berlin52.tsp:

Balance	Total cost	Minimax score	Route 2 cost	Route 1 cost	Configuration
71	23673	11872	11872	11801	regular_ga
18	20676	10347	10329	10347	partial_memetic
6	19370	9688	9682	9688	full_memetic

• eil51.tsp:

Balance	Total cost	Minimax score	Route 2 cost	Route 1 cost	Configuration
2	1320	661	661	659	regular_ga
0	1100	550	550	550	partial_memetic
7	1047	527	520	527	full_memetic

:Att48.tsp •

Balance	Total cost	Minimax score	Route 2 cost	Route 1 cost	Configuration
168	33102	16635	16635	16467	regular_ga
42	26682	13362	13362	13320	partial_memetic
116	25802	12959	12959	12843	full_memetic

:dantzig42.tsp •

Balance	Total cost	Minimax score	Route 2 cost	Route 1 cost	Configuration
10	2124	1067	1067	1057	regular_ga
8	1894	951	943	951	partial_memetic
6	1812	909	909	903	full_memetic

:bayg29.tsp •

Balance	Total cost	Minimax score	Route 2 cost	Route 1 cost	Configuration
8	3964	1986	1986	1978	regular_ga
9	3955	1982	1982	1973	partial_memetic
5	3783	1894	1889	1894	full_memetic

:fri26.tsp •

Balance	Total cost	Minimax score	Route 2 cost	Route 1 cost	Configuration
8	2364	1186	1178	1186	regular_ga
16	2326	1171	1155	1171	partial_memetic
26	2274	1150	1150	1124	full_memetic

:gr24.tsp •

Balance	Total cost	Minimax score	Route 2 cost	Route 1 cost	Configuration
42	3264	1653	1653	1611	regular_ga
1	3379	1690	1689	1690	partial_memetic
24	3226	1625	1625	1601	full_memetic

:gr17.tsp •

Balance	Total cost	Minimax score	Route 2 cost	Route 1 cost	Configuration
7	5041	2524	2517	2524	regular_ga
63	5137	2600	2537	2600	partial_memetic
28	4934	2481	2481	2453	full_memetic

:burma14.tsp •

Balance	Total cost	Minimax score	Route 2 cost	Route 1 cost	Configuration
68	7566	3817	3817	3749	regular_ga
15	7639	3827	3812	3827	partial_memetic
15	7555	3785	3785	3770	full_memetic

הפליטים וגרפים עבור כל קובץ קלט מצורפים בתיקיית results כתבתי את הטבלאות האלה עבור כל קלט לפשטות במקום להראות הדברים המיותרת.